

Assignment 1 Solution (DDL: 2025/10/10)

1. Vector Calculus Review (15 pts)

Let $x, c \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. For the following parts, before taking any derivatives, identify what the derivative looks like (is it a scalar, vector, or matrix?) and how we calculate each term in the derivative. Then carefully solve for an arbitrary entry of the derivative, then stack/arrange all of them to get the final result. Note that the convention we will use going forward is that vector derivatives of a scalar (with respect to a column vector) are expressed as a row vector, i.e. $\frac{\partial f}{\partial x} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$ since a row acting on a column gives a scalar. You may have seen alternative conventions before, but the important thing is that you need to understand the types of objects and how they map to the shapes of the multidimensional arrays we use to represent those types.

(a) Show $\frac{\partial}{\partial x} (x^T c) = c^T$

Solution: This is a vector derivative of a scalar quantity, so our result will be a row vector. Looking at the i -th entry, $\frac{\partial}{\partial x_i} (x^T c) = \frac{\partial}{\partial x_i} (\sum_j c_j x_j) = c_i$. Stacking all the entries into a row vector, we get c^T .

(b) Show $\frac{\partial}{\partial x} \|x\|_2^2 = 2x^T$

Solution: This is a vector derivative of a scalar quantity, so our result will be a row vector. Looking at the i -th entry, $\frac{\partial}{\partial x_i} (\|x\|_2^2) = \frac{\partial}{\partial x_i} (\sum_j x_j^2) = 2x_i$. Stack all the entries into a row to get $2x^T$.

(c) Show $\frac{\partial}{\partial x} (Ax) = A$

Solution: This is a vector derivative of a vector quantity, so the result will be a matrix. Let $f = Ax$. Note that $f_i = \sum_k A_{ik} x_k$

Looking at the (i, j) -th entry of our matrix, $\frac{\partial f_i}{\partial x_j} = \frac{\partial}{\partial x_j} (\sum_k A_{ik} x_k) = A_{ij}$. Arranging all of these in a matrix will recover A .

(d) Show $\frac{\partial}{\partial x} (x^T Ax) = x^T (A + A^T)$

Solution: This is a vector derivative of a scalar quantity, so our result will be a vector. Before taking any derivatives, we can write $x^T Ax = \sum_i \sum_j A_{ij} x_i x_j$. Taking the derivative with respect to an arbitrary x_k and focusing on just the terms involving x_k (as the derivative of the other terms wrt x_k is zero), we can write

$$\begin{aligned} \frac{\partial}{\partial x_k} (x^T Ax) &= \frac{\partial}{\partial x_k} \left(\left(\sum_{j \neq k} A_{kj} x_k x_j \right) + \left(\sum_{i \neq k} A_{ik} x_i x_k \right) + A_{kk} x_k^2 \right) \\ &= \left(\sum_{j \neq k} A_{kj} x_j \right) + \left(\sum_{i \neq k} A_{ik} x_i \right) + 2A_{kk} x_k \\ &= \left(\sum_j A_{kj} x_j \right) + \left(\sum_i A_{ik} x_i \right) = \sum_i (A_{ki} x_i + A_{ik} x_i) \\ &= x^T (\text{kth row of } A + \text{kth col of } A) = x^T (A_k + A_k^T) \end{aligned}$$

Stacking all the results in a row vector, we get $\frac{\partial}{\partial x} (x^T Ax) = x^T (A + A^T)$ as desired.

(e) Under what condition is the previous derivative equal to $2x^T A$?

Solution: We want $(A + A^T) = 2A$. This is true if and only if $A = A^T$, i.e. the matrix A is symmetric.

2. Bayes' Rule (10 pts)

Assume the probability of a certain disease is 0.01. The probability of test positive given that a person is infected with the disease is 0.95 and the probability of test positive given the person is not infected with the disease is 0.05.

(a) Calculate the probability of test positive.

(b) Use Bayes' Rule to calculate the probability of being infected with the disease given that the test is positive.

Solution:

(a) Given the information in the problem, we have $P(D) = 0.01$, $P(T|D) = 0.95$ and $P(T|\bar{D}) = 0.05$. Thus, we have $P(T) = P(T \cap D) + P(T \cap \bar{D}) = P(T|D)P(D) + P(T|\bar{D})P(\bar{D}) = 0.95 \times 0.01 + 0.05 \times 0.99 = 0.059$.

(b) $P(D|T) = P(T|D)P(D)/P(T) = 0.95 \times 0.01/0.059 = 0.16$

3. Gradient Descent Mechanics (20 pts)

Gradient descent is the primary algorithm to search optimal parameters for our models. Typically, we want to solve optimization problems stated as

$$\min_{\theta \in \Theta} \mathcal{L}(f_{\theta}, \mathcal{D})$$

where $\mathcal{L}$ are differentiable functions. In this example, we look at a simple supervised learning problem where given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, we want to find the optimal parameters θ that minimizes some loss. We consider different models for learning the mapping from input to output, and examine the behavior of gradient descent for each model.

(a) The simplest parametric model entails learning a single-parameter constant function, where we set $\hat{y}_i = \theta$. We wish to find

$$\hat{\theta}_{const} = \min_{\theta \in \mathbb{R}} \mathcal{L}(f_{\theta}, \mathcal{D}) = \min_{\theta \in \mathbb{R}} \frac{1}{N} \sum_{i=1}^N (y_i - \theta)^2$$

i. What is the gradient of $\mathcal{L}$ with respect to θ ?

ii. What is the optimal value of θ ?

iii. Write the gradient descent update.

iv. Stochastic Gradient Descent (SGD) is an alternative optimization algorithm, where instead of using all N samples, we use single sample per optimization step to update the model. What is the contribution of each data-point to the full gradient update?

Solution:

- Taking the gradient of $\mathcal{L}$ w.r.t θ we have

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{2}{N} \sum_{i=1}^N (\theta - y_i)$$

- From the first-order optimality condition, at θ^* we have $\frac{\partial \mathcal{L}}{\partial \theta} = 0$. Using this, we get

$$\begin{aligned} \frac{2}{N} \sum_{i=1}^N (\theta^* - y_i) &= 0 \\ \implies \theta^* &= \frac{1}{N} \sum_{i=1}^N y_i \end{aligned}$$

- The gradient descent update follows the rule

$$\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$$

where α is the step-size. Plugging in the gradient, we have

$$\theta \leftarrow \theta - \frac{2\alpha}{N} \sum_{i=1}^N (\theta - y_i)$$

- Using SGD, we update the model by uniformly sampling a single data-point per iteration, with the following update rule

$$\theta \leftarrow \theta - \frac{2\alpha}{N} (\theta - y_i)$$

The full gradient can be written as

$$\nabla_{\theta} \mathcal{L} = \frac{2}{N}(\theta - y_1) + \frac{2}{N}(\theta - y_2) + \frac{2}{N}(\theta - y_3) + \cdots + \frac{2}{N}(\theta - y_N)$$

which suggests that each data-point contributes equally to the full gradient.

- (b) Instead of constant functions, we now consider a single-parameter linear model $\hat{y}_i(x_i) = \theta(x_i)$ where we search for θ such that

$$\hat{\theta} = \min_{\theta \in \mathbb{R}} \frac{1}{N} \sum_{i=1}^N (y_i - \theta x_i)^2$$

- What is the gradient of $\mathcal{L}$ with respect to θ ?
- What is the optimal value of θ ?
- Write the gradient descent update.
- Do all points get the same vote in the update? Why or why not?

Solution:

- Taking the gradient of $\mathcal{L}$ w.r.t θ we have

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{2}{N} \sum_{i=1}^N (\theta x_i - y_i) x_i$$

- From the first-order optimality condition, at θ^* we have $\frac{\partial \mathcal{L}}{\partial \theta} = 0$. Using this, we get

$$\begin{aligned} \frac{2}{N} \sum_{i=1}^N (y_i - \theta x_i) x_i &= 0 \\ \implies \theta^* &= \frac{\sum_{i=1}^N x_i y_i}{\sum_{i=1}^N x_i^2} \end{aligned}$$

- The gradient descent update follows the rule

$$\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$$

where α is the step-size. Plugging in the gradient, we have

$$\theta \leftarrow \theta - \frac{2\alpha}{N} \sum_{i=1}^N (\theta x_i - y_i) x_i$$

- Using SGD, we update the model by uniformly sampling a single data-point per iteration, with the following update rule

$$\theta \leftarrow \theta - \frac{2\alpha}{N} (\theta x_i - y_i) x_i$$

The full gradient can be written as

$$\nabla_{\theta} \mathcal{L} = \frac{2}{N} (\theta x_1 - y_1) x_1 + \frac{2}{N} (\theta x_2 - y_2) x_2 + \frac{2}{N} (\theta x_3 - y_3) x_3 + \cdots + \frac{2}{N} (\theta x_N - y_N) x_N$$

The contribution of each data-point to the full gradient depends on the size of the data-point x_i , i.e. data-points closer to the origin contribute less to the full gradient.

4. MAP Interpretation of Ridge Regression (20 pts)

Consider the Ridge Regression estimator (Hint: here the X is feature matrix, equivalent to X^T in our slides)

$$\arg \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2$$

We know this is solved by

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

One interpretation of Ridge Regression is to find the Maximum A Posteriori (MAP) estimate on $\mathbf{w}$, the parameters, assuming that the prior of $\mathbf{w}$ is $N(0, \mathbf{I})$ and that the random $\mathbf{Y}$ is generated using

$$\mathbf{Y} = \mathbf{X}\mathbf{w} + \sqrt{\lambda} \mathbf{N}$$

Note that each entry of vector $\mathbf{N}$ is zero-mean, unit-variance normal. Show that $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$ is indeed the MAP estimate for $\mathbf{w}$ given an observation on $\mathbf{Y} = \mathbf{y}$.

Solution: From how we define MAP estimation,

$$\begin{aligned} MAP(\mathbf{w}|\mathbf{Y} = \mathbf{y}) &= \operatorname{argmax}_{\mathbf{w}} f(\mathbf{w}|\mathbf{Y} = \mathbf{y}) \\ &= \operatorname{argmax}_{\mathbf{w}} \frac{f(\mathbf{w}, \mathbf{y})}{f(\mathbf{y})} \end{aligned}$$

The denominator doesn't affect the argmax since it doesn't depend on $\mathbf{w}$, so we can omit it. Then we can use the chain rule to expand out the numerator

$$= \operatorname{argmax}_{\mathbf{w}} f(\mathbf{w})f(\mathbf{y}|\mathbf{w})$$

Now, split up the conditional joint density into the product of conditional densities since each element of the $\mathbf{y}$ is independent given $\mathbf{w}$.

$$= \operatorname{argmax}_{\mathbf{w}} f(\mathbf{w}) \prod_{i=1}^n f(y_i|\mathbf{w})$$

We can now recall the formula for standard normal pdf is $f_Z(z) = \frac{e^{-z^2/2}}{\sqrt{2\pi}}$. To find $f(y_i|\mathbf{w})$, we know that $y_i = \mathbf{x}_i^T \mathbf{w} + \sqrt{\lambda} N_i$, where $N_i \sim \mathcal{N}(0, 1)$, so we'd have $\frac{y_i - \mathbf{x}_i^T \mathbf{w}}{\sqrt{\lambda}} \sim \mathcal{N}(0, 1)$. Now, plugging in the pdf in the previous expressions we'd have

$$MAP = \operatorname{argmax}_{\mathbf{w}} \frac{e^{-\|\mathbf{w}\|^2/2}}{\sqrt{2\pi}} \prod_{i=1}^n \frac{e^{-(\frac{y_i - \mathbf{x}_i^T \mathbf{w}}{\sqrt{\lambda}})^2/2}}{\sqrt{2\pi}}$$

We can ignore multiplicative scaling constants (since they don't affect the value of the argmax. Also, since log is a monotonically increasing function, we can take log of both sides without affecting the argmax. Taking logs is useful since it allows us to turn products into sums. So we now have:

$$MAP = \operatorname{argmax}_{\mathbf{w}} -\frac{\|\mathbf{w}\|^2}{2} - \frac{1}{\lambda} \sum_i \frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{2}$$

We can ignore scaling factors again, and arrange all the summation terms in a vector and taking the norm-squared. We can also turn argmax into argmin by negating the objective function:

$$MAP = \operatorname{argmin}_{\mathbf{w}} \|\mathbf{w}\|^2 + \frac{1}{\lambda} \|\mathbf{y} - X\mathbf{w}\|^2$$

Finally, we can multiply through by λ without changing the argmax since it is a positive constant:

$$MAP = \operatorname{argmax}_{\mathbf{w}} \lambda \|\mathbf{w}\|^2 + \|X\mathbf{w} - \mathbf{y}\|^2$$

And now it is the same form as the original Ridge Regression optimization problem.

5. Programming (35 pts)

In this lab, please write your code in the word file directly.